(a) Specify the decision steps that you use in your algorithm design.

(b) Specify alternatives at each decision step.

(c) What is the greedy choice that you use while selecting one alternative in your algorithm?

(d) Why this greedy choice may lead to an optimal solution?

(e) Design a greedy algorithm for the problem (You should give a pseudocode).

(f) Analysis time and space complexity of your algorithm.

(g) Does your algorithm always provide an optimal solution? Prove.

Question 4.1

(a) Specify the decision steps that you use in your algorithm design.

(Solution) The decision steps of the algorithm are given in Figure 1.

(b) Specify alternatives at each decision step.

(Solution) The alternatives while applying the decision steps in Figure 1 as follows.

- At the 1^{st} step, the alternatives are the triples in arr. Assume that cardinality of arr is n. Then, the triples are $arr[0], arr[1], arr[2]$, $arr[1], arr[2], arr[3]$, ..., $arr[n-3], arr[n-2], arr[n-1]$.
- At the 3^{rd} step, the alternatives are the shape types. For instance, if a triple whose elements have the same value, then this triple can be valley or peak.

(c) What is the greedy choice that you use while selecting one alternative in your algorithm?

(Solution) The greedy choice of the problem is to annotate the relation between each triple in the set to distribute the minimum amount of money to the set.

(d) Why this greedy choice may lead to an optimal solution?

(Solution) Assume that we select each pair element in the set instead of selecting triples at the first step in Figure 1. Then the relation can be annotated only with **rise** and **fall**. Since we cannot decide backward or forward distribution with the pair element selection, the amount of money may be increased. Hence, we annotate the relations between each triple elements in the set to avoid it.

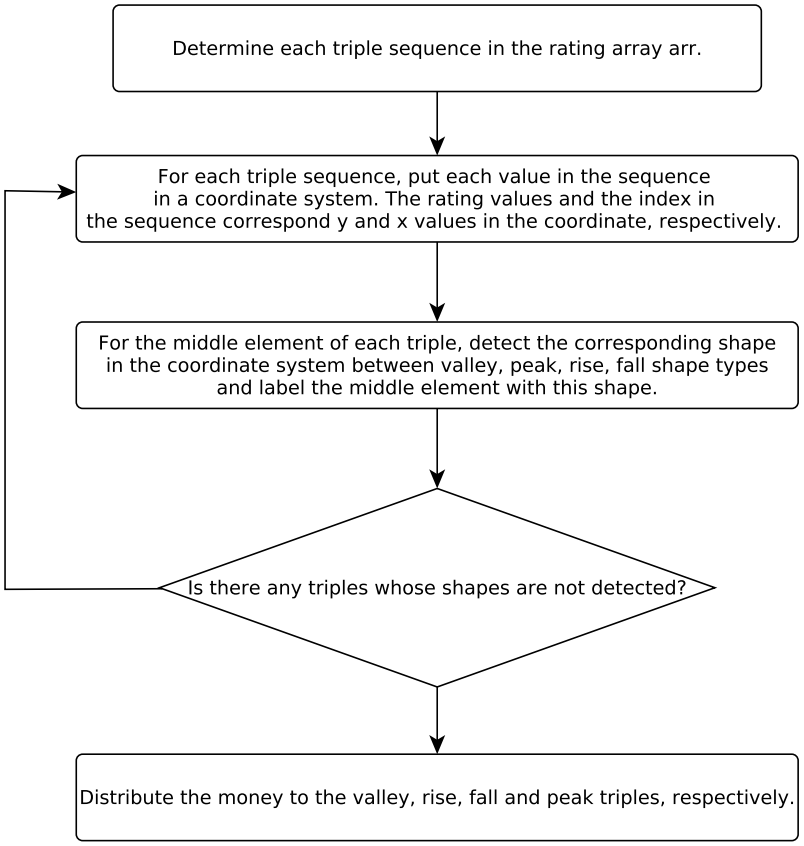

Figure 1: Decision Steps of The Algorithm

(e) Design a greedy algorithm for the problem (You should give a pseudocode).

(Solution) We denote by E_i the amount of money given to the i^{th} employee.
Let's classify the employees into four kinds, depending on the comparisons of their ratings with their neighbors.

- If $rating_{i-1} \geq rating_i \leq rating_{i+1}$, we say Employee i a **valley**.
- If $rating_{i-1} < rating_i \leq rating_{i+1}$, we say Employee i a **rise**.
- If $rating_{i-1} \geq rating_i > rating_{i+1}$, we say Employee i a **fall**.
- If $rating_{i-1} < rating_i > rating_{i+1}$, we say Employee i a **peak**.

For the leftmost and rightmost employee, assume they each have a neighbor with an infinite rating, so they can also be classified according to this scheme. (In particular, the leftmost employee cannot be a rise or a peak, and the rightmost employee cannot be a fall or a peak.)

Given this classification, we can intuitively distribute money this way:

- For each valley employee, give him/her \$100.
- For each rise employee, give him/her E_{i-1} + \$100.
- For each fall employee, give him/her E_{i+1} + \$100.
- For each peak employee, give him/her $\max(E_{i-1}, E_{i+1})$ + \$100.

Observe that this gives a valid distribution of money. But does it give the minimum amount of money? Yes, but only as long as you distribute the money in the right order. Here's one good order:

- Distribute the money to the valleys.
- Distribute the money to the rises from left to right.
- Distribute the money to the falls from right to left.
- Distribute the money to the peaks.

Note that the order in which we distribute money to rises and falls is important!

(f) Analysis time and space complexity of your algorithm.

- Finding k triples in the set: O(n).
- Detect the shapes of k triples: O(k)
- Distribute money to k triples: O(k)

Therefore, the time complexity O(n) and there is no exceed for the memory to keep data more than n and the space complexity is O(1).

(g) Does your algorithm always provide an optimal solution? Prove.

(Solution) There are two things we need to prove:

- It gives a valid distribution of money.
- It gives a distribution with the minimum amount of money.

Is it valid?

The first thing to notice is that: All employees are assigned a positive amount of money. We just have to show that for every two neighbors with different ratings, the one with the higher rating is given more money. Consider two neighboring employees and with different ratings. There are two cases:

- Case 1: $rating_i < rating_{i+1}$
  In this case, the only possible cases are:
  - Employee i is a valley and Employee $i+1$ is a peak.
  - Employee i is a valley and Employee $i+1$ is a rise.
  - Employee i is a rise and Employee $i+1$ is a peak.
  - Employee i is a rise and Employee $i+1$ is a rise.

  In the first three cases, we can see that Employee i is given money earlier than Employee $i+1$ (due to the order we're giving out money), and indeed, we see that $E_i < E_{i+1}$. But actually, the same holds as well in the last case, since we're giving money to rises from left to right.

- Case 2: $rating_i > rating_{i+1}$
  This is similar. In this case, the only possible cases are:
  - Employee i is a peak and Employee $i+1$ is a valley.
  - Employee i is a fall and Employee $i+1$ is a valley.
  - Employee i is a peak and Employee $i+1$ is a fall.
  - Employee i is a fall and Employee $i+1$ is a fall.

  In the first three cases, we can see that Employee $i+1$ is given money earlier than Employee i , and indeed, we see that $E_i > E_{i+1}$. But actually, the same holds as well in the last case, since we're giving money to falls from right to left.

This shows that the money distribution is valid for every pair of neighbors, hence the overall distribution is valid as well.

Is it minimum?

Proving that this gives the minimum total money is a little easier. We just have to notice that these inequalities hold:

- For each valley Employee i, $E_i \geq \$100$.
- For each rise Employee i, $E_i \geq E_{i-1} + \$100$.
- For each fall Employee i, $E_i \geq E_{i+1} + \$100$.
- For each peak Employee i, $E_i \geq max(E_{i-1}, E_{i+1}) + \$100$.

This easily follows from the constraints of the problem. Since the algorithm above satisfies each inequality in the tightest possible way, this means the overall amount of money is minimized.

(h) Apply the following instances to your algorithm (Show your work step by step):

- $n = 10$
- $arr = [2, 4, 2, 6, 1, 7, 8, 9, 2, 1]$

(Solution)

- $rating_0 < rating_1 > rating_2$, so Employee 1 is a **peak**.
- $rating_1 > rating_2 < rating_3$, so Employee 2 is a **valley**.
- $rating_2 < rating_3 > rating_4$, so Employee 3 is a **peak**.
- $rating_3 > rating_4 < rating_5$, so Employee 4 is a **valley**.
- $rating_4 < rating_5 < rating_6$, so Employee 5 is a **rise**.
- $rating_5 < rating_6 < rating_7$, so Employee 6 is a **rise**.
- $rating_6 < rating_7 > rating_8$, so Employee 7 is a **peak**.
- $rating_7 > rating_8 > rating_9$, so Employee 8 is a **fall**.

Given this classification, we can intuitively distribute money this way:

- Employee 2 and 4 take \$100 because they are valley.
- Employee 5 takes Employee 4 + \$100 = 200\$ because it is rise. Employee 6 takes Employee 5 + \$100 = 300\$ because it is rise.
- Employee 9 takes \$100 with the rightmost rule. Employee 8 takes Employee 9+\$100 = \$200 because it is fall.
- Employee 3 takes $\max(E_2, E_4)$+\$100 = \$200 because it is peak. Employee 1 takes $\max(E_0, E_2)$+\$100 = \$200 because it is peak. Hence Employee 0 takes \$100. Employee 7 takes $\max(E_6, E_8)$+\$100 = \$400 because it is peak.

(i) What is the distribution of money that John must pay for the given instances in (h) when you apply your algorithm?

(Solution) The result is [100, 200, 100, 200, 100, 200, 300, 400, 200, 100].

(j) What is the total amount of money that John must pay for the given instances in (h)?

(Solution) 100 + 200 + 100 + 200 + 100 + 200 + 300 + 400 + 200 + 100 = \$1900.

Question 4.2

(a) Specify the decision steps that you use in your algorithm design.

(Solution) The decision steps of the algorithm are given in Figure 2.

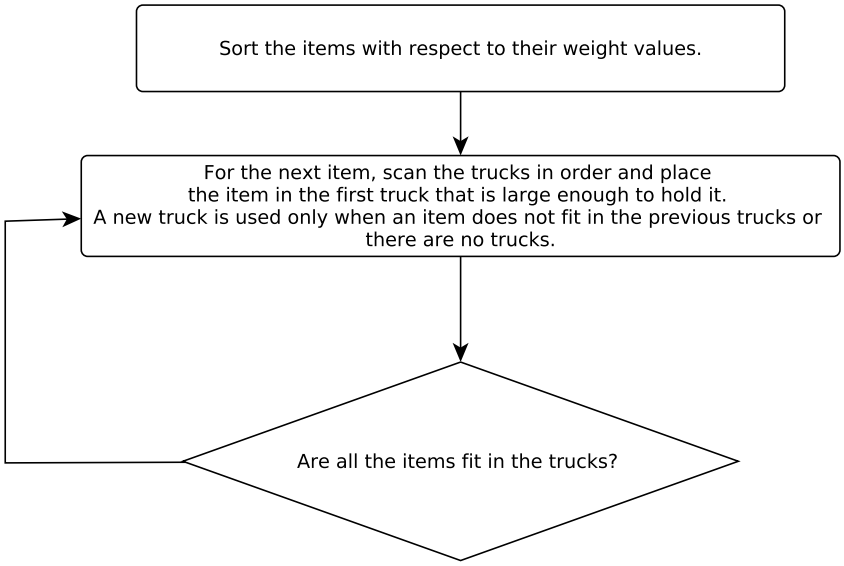

Figure 2: Decision Steps of The Algorithm for Problem 2

(b) Specify alternatives at each decision step.

(Solution) In the second step of Figure 2, the existing trucks are alternatives to choose and we should select the first truck which is available to fit the next time according to its remaining weight.

(c) What is the greedy choice that you use while selecting one alternative in your algorithm?

(Solution) Scan the trucks in an order and place the new item in the first truck that is large enough to hold it. A new truck is used only when an item does not fit in the previous trucks.

(d) Why this greedy choice may lead to an optimal solution?

(Solution) To choose the first truck which is large enough to fit the next item from the sorted set leads to utilize the existing trucks.
Assume that an item is selected randomly from the set to fit the first truck which is large enough for the item. For instance, there are two trucks available with their remaining weight values $b_1 = 5$ and $b_2 = 3$ and two items with weights $w_1 = 4$ and $w_2 = 3$. When w_2 is selected randomly to fit to b_1, w_1 cannot be fit. However, if the items are selected according to their weights then the first truck which is large enough always has the largest remaining weight value.

(e) Design a greedy algorithm for the problem (You should give a pseudocode).

(Solution)

(f) Analysis time and space complexity of your algorithm.

(Solution) The algorithm is easily implemented in $O(n^2)$ time. However it also can be implemented in O(nlogn)

Algorithm 1: Pseudo Code of The Algorithm

```
Input: w_arr = [w_1, w_2, ..., w_n], n, M
// Initialize result (Count of bins)
res = 0
// Create an array to store remaining space in bins
// there can be at most n bins
bin_rem = [ ]
//Sort w_arr with respect to their weight values in descending order
w_arr = sort(w_arr)
// Place items one by one
for i ← 1 to n do
    // Find the first bin that can accommodate w_arr[i]
    for j ← 1 to res do
        if bin_rem[j] ≥ w_arr[i] then
            bin_rem[j] = bin_rem[j] - w_arr[i]
            break
        end
    end
    // If no bin could accommodate w_arr[i]
    if j == res then
        bin_rem[res] = M - w_arr[i]
        res++
    end
end
return res
```

as follows:
Idea: Use a balanced search tree with height O(logn).
Each node has three values: index of truck, remaining capacity of truck, and best (largest) in all the bins represented by the subtree rooted at the node.
The ordering of the tree is by truck index.

(g) Does your algorithm always provide an optimal solution? Prove.

(Solution) Assume that OPT: # trucks used in the optimal solution.
Suppose that Algorithm 1 uses m trucks.
Then, at least $(m-1)$ trucks are more than half full.
We never have two trucks less than half full.
If there are two trucks less than half full, items in the second truck can be substituted into the first truck by the algorithm.

- OPT $\geq \frac{\sum_{i=1}^{n} w_i}{W} > \frac{m-1}{2}$
- 2OPT > m-1
- 2OPT $\geq$ m Since m and OPT are integers.

(h) Apply your algorithm to the following instances (Show your work step by step):

- There are 9 items and their weight values are $w_1 = 5$, $w_2 = 7$, $w_3 = 3$, $w_4 = 5$, $w_5 = 12$, $w_6 = 11$, $w_7 = 10$, $w_8 = 11$, $w_9 = 9$.
- The capacity value of each truck: $M = 14$.

(Solution)

- Sort the items based on weight values $\rightarrow w_5, w_6, w_8, w_7, w_9, w_2, w_1, w_4, w_3$.

- w_5 goes to the first bin b_1.
- w_6 goes to the second bin b_2 since the remaining weight of b_1 is not enough for w_6.
- w_8 goes to the third bin b_3 since the remaining weight of b_1 and b_2 are not enough for w_8.
- w_7 goes to the forth bin b_4 since the remaining weight of b_1, b_2 and b_3 are not enough for w_7.
- w_9 goes to the fifth bin b_5 since the remaining weight of b_1, b_2, b_3 and b_4 are not enough for w_9.
- w_2 goes to the sixth bin b_6 since the remaining weight of b_1, b_2, b_3, b_4 and b_5 are not enough for w_2.
- w_4 goes to b_5 since the remaining weight of the bin is 5 which is equal to w_4.
- w_3 goes to b_2 since the remaining weight of the bin is 3 which is equal to w_3.

(i) How many trucks are used to fit all the items in (h)?
(Solution) 6 trucks are used to fit all the items in (h)

(j) Does your algorithm give the optimal solution for the given instances in (h)?
(Solution)
Recall (g): OPT $\geq \sum_{i=1}^{n} w_i > \frac{m-1}{2}$

$6 \geq \frac{12+11+11+10+9+7+5+3}{14} > \frac{5-1}{2}$

$6 \geq 4.86 > 2$ so the algorithm gives the optimal solution.